

TCL SCRIPTING

for Design Verification

Topics Covered

Section 1 : File I/O & Log Parsing

Section 2 : Arrays

Section 3 : Procs (Procedures)

Section 4 : List Operations

Section 5 : String Operations

SECTION 1 : File I/O & Log Parsing

Q1 Read & Count Lines in a Log File

Write a TCL script that opens a simulation log file, counts total lines, and prints the count. Handle missing files gracefully.

Key Concepts: file exists, open, gets, close, error handling

file exists	Checks whether a file path exists before opening
open / close	Opens a file handle for reading; always close after use
gets \$fh line	Reads one line at a time; returns -1 at EOF

TCL Script:

```
set filename "sim.log"

if {[file exists $filename]} {
    puts "ERROR: File $filename not found!"
    exit 1
}

set fh [open $filename r]
set count 0

while {[gets $fh line] != -1} {
    incr count
}
close $fh

puts "Total lines in $filename : $count"
```

Sample Output:

```
Total lines in sim.log : 248
```

Q2 Parse Errors & Warnings from Log

Write a TCL script that reads a simulation log, collects all lines containing *ERROR* or *WARNING* (case-insensitive), and writes them to a report file with line numbers.

Key Concepts: string tolower, regexp, format, file write

string tolower	Converts string to lowercase for case-insensitive comparison
regexp	Pattern match to detect ERROR/WARNING keywords in a line
format	Formats output strings with padding and alignment

TCL Script:

```

set infile "sim.log"
set outfile "error_report.txt"

set rfh [open $infile r]
set wfh [open $outfile w]

set linenum 0
set errcount 0

puts $wfh "======"
puts $wfh " Simulation Error & Warning Report  "
puts $wfh "======"

while {[gets $rfh line] != -1} {
    incr linenum
    set lower [string tolower $line]
    if {[regexp {error|warning} $lower]} {
        puts $wfh "[format {Line %4d} $linenum] : $line"
        incr errcount
    }
}

puts $wfh "======"
puts $wfh " Total Issues Found : $errcount"
puts $wfh "======"

close $rfh
close $wfh
puts "Report written to $outfile ($errcount issues found)"

```

Sample Output:

```
Report written to error_report.txt (5 issues found)
```

error_report.txt content:

```

====
Simulation Error & Warning Report
====
Line   12 : WARNING: clk not driven at time 0
Line   47 : ERROR: Assertion failed req_ack_check
Line  102 : WARNING: X-state detected on data bus
====
Total Issues Found : 3
=====

```

Q3 Extract Simulation Timestamps

Write a TCL script that reads a log file and extracts all simulation timestamps (lines starting with '#' followed by a number, e.g. '#100'). Print the first, last, and total count.

Key Concepts: regexp with capture groups, lappend, lindex, llength

regexp -inline	Returns matched substrings directly without needing a variable
lappend	Appends elements to the end of a list
lindex \$l end	Retrieves the last element using the 'end' keyword

TCL Script:

```
set filename "sim.log"
set fh      [open $filename r]
set timestamps {}

while {[gets $fh line] != -1} {
    if {[regexp {^#(\d+)} $line match ts]} {
        lappend timestamps $ts
    }
}
close $fh

set total [llength $timestamps]

if {$total == 0} {
    puts "No timestamps found."
} else {
    puts "Total timestamps : $total"
    puts "First timestamp  : [lindex $timestamps 0] ns"
    puts "Last timestamp   : [lindex $timestamps end] ns"
}
```

Sample Output:

```
Total timestamps : 120
First timestamp  : 0 ns
Last timestamp   : 5000 ns
```

SECTION 2 : Arrays

Q4 Signal State Tracker using Array

Write a TCL script that uses an array to track the current state (HIGH/LOW/X/Z) of multiple signals. Provide a procedure to update a signal's state and another to display the full state table.

Key Concepts: array set, array names, upvar, format

array set	Initializes an array with key-value pairs in one statement
upvar	Links a proc variable to a variable in the caller's scope
array names	Returns all keys of an array; use lsort to sort them

TCL Script:

```

array set sig_state {
    clk      HIGH
    reset    LOW
    data     X
    valid    LOW
    ready    Z}

proc update_signal {arr_name signal new_state} {
    upvar $arr_name arr
    if {[info exists arr($signal)]} {
        set arr($signal) $new_state
        puts "Updated: $signal => $new_state"
    } else {
        puts "ERROR: Signal $signal not found!"
    }
}

proc show_states {arr_name} {
    upvar $arr_name arr
    puts "======"
    puts " Signal          State"
    puts "======"
    foreach sig [lsort [array names arr]] {
        puts " [format %-16s $sig] $arr($sig)"
    }
    puts "======"
}

show_states sig_state
update_signal sig_state data HIGH
update_signal sig_state valid HIGH
puts ""
show_states sig_state

```

Sample Output:

```

=====
Signal          State
=====
clk             HIGH
data            X
ready           Z
reset           LOW
valid           LOW
=====
Updated: data => HIGH
Updated: valid => HIGH

=====
clk             HIGH
data            HIGH
ready           Z
reset           LOW
valid           HIGH
=====

```

Q5 Multi-Run Regression Tracker using Array

Write a TCL script that stores test results across 3 regression runs using a 2D array (indexed by run and test name). After all runs, print a comparison table and flag tests that changed status.

Key Concepts: 2D array keys (run,test), array names, lsort -unique

arr(run,test)	2D array using comma-separated key e.g. results(run1,test_reset)
array names -glob	Filters array keys using a glob pattern
lsort -unique	Sorts a list and removes all duplicate entries

TCL Script:

```

array set results {
    run1,test_reset    PASS
    run1,test_read     FAIL
    run1,test_write    PASS
    run1,test_overflow FAIL
    run2,test_reset    PASS
    run2,test_read     PASS
    run2,test_write    PASS
    run2,test_overflow FAIL
    run3,test_reset    PASS
    run3,test_read     PASS
    run3,test_write    FAIL
    run3,test_overflow PASS
}

set tests {test_reset test_read test_write test_overflow}
set runs  {run1 run2 run3}

puts "[format %-18s Test]  run1    run2    run3"

```

```
puts "-----"

foreach test $tests {
    set row "[format %-18s $test]"
    set statuses {}
    foreach run $runs {
        set s $results($run,$test)
        lappend statuses $s
        append row " [format %-6s $s]"
    }
    if {[llength [lsort -unique $statuses]] > 1} {
        append row " <-- CHANGED"
    }
    puts $row
}
```

Sample Output:

Test	run1	run2	run3	
test_reset	PASS	PASS	PASS	
test_read	FAIL	PASS	PASS	<-- CHANGED
test_write	PASS	PASS	FAIL	<-- CHANGED
test_overflow	FAIL	FAIL	PASS	<-- CHANGED

Q6 Array Statistics — Min, Max, Average

Write a TCL script that stores simulation timing data (signal transition times) in an array. Write a proc to compute the minimum, maximum, and average values from all array entries and display a report.

Key Concepts: array get, foreach over array, expr double(), format

array get	Returns the array as a flat key-value list for iteration
array size	Returns the number of elements currently in the array
expr double()	Converts integer to float for accurate division in avg calculation

TCL Script:

```
# Transition times in nanoseconds
array set trans_times {
    clk_rise_1    10
    clk_rise_2    30
    data_change   45
    valid_assert  50
    ack_assert    78
    clk_rise_3    60
    reset_deassert 20
}

proc array_stats {arr_name} {
    upvar $arr_name arr
    set vals {}
    foreach key [array names arr] {
        lappend vals $arr($key)
    }
}
```

```

    }
    set vals [lsort -integer $vals]
    set min [lindex $vals 0]
    set max [lindex $vals end]
    set sum 0
    foreach v $vals { set sum [expr {$sum + $v}] }
    set avg [expr {double($sum) / [llength $vals]}]
    set cnt [array size arr]

    puts "====="
    puts " Array Statistics Report"
    puts "====="
    puts " Total Entries : $cnt"
    puts " Minimum Value : $min ns"
    puts " Maximum Value : $max ns"
    puts " Average Value : [format %.2f $avg] ns"
    puts " Sum Total      : $sum ns"
    puts "====="
}

array_stats trans_times

```

Sample Output:

```

=====
Array Statistics Report
=====
Total Entries : 7
Minimum Value : 10 ns
Maximum Value : 78 ns
Average Value : 41.86 ns
Sum Total      : 293 ns
=====

```

SECTION 3 : Procs (Procedures)

Q7 Proc with Default Arguments & Error Handling

Write a TCL proc called 'run_test' that accepts a test name, an optional seed (default 42), and an optional timeout (default 100). Validate inputs, handle errors with catch, and return a formatted result string.

Key Concepts: default args, error, catch, return

default args	Syntax: proc name {arg {opt default}} — optional args have default values
error	Throws a runtime error with a custom message from inside a proc
catch	Catches runtime errors without crashing; stores message in a variable

TCL Script:

```
proc run_test {testname {seed 42} {timeout 100}} {
    if {[string length $testname] == 0} {
        error "Test name cannot be empty!"
    }
    if {$timeout <= 0} {
        error "Timeout must be a positive integer!"
    }
    # Odd seed = FAIL, Even seed = PASS
    set status [expr {$seed % 2 == 0 ? "PASS" : "FAIL"}]
    return "[format %-20s $testname] | Seed:[format %4d $seed] | Timeout:[format
%4d $timeout]ns | $status"
}

# All arguments provided
puts [run_test "test_alu_add" 101 200]
puts [run_test "test_mem_write" 42 500]

# Only test name - use both defaults
puts [run_test "test_reset"]

# Test name + seed - timeout uses default
puts [run_test "test_overflow" 7]

# Error handling
catch {run_test "" 10 50} err
puts "Caught Error: $err"

catch {run_test "test_x" 5 -1} err
puts "Caught Error: $err"
```

Sample Output:

```

test_alu_add      | Seed: 101 | Timeout: 200ns | FAIL
test_mem_write    | Seed:  42 | Timeout: 500ns | PASS
test_reset        | Seed:  42 | Timeout: 100ns | PASS
test_overflow     | Seed:   7 | Timeout: 100ns | FAIL
Caught Error: Test name cannot be empty!
Caught Error: Timeout must be a positive integer!

```

Q8 Proc Returning Multiple Values via List

Write a TCL proc called 'signal_analysis' that takes a list of signal transition times, and returns minimum, maximum, average, and count as a structured list. Use lassign to unpack the result in the caller.

Key Concepts: return list, lassign, lsort -integer, llength

return list	A proc can return a list to pass multiple values to the caller
lassign	Unpacks a list into individual variables: lassign \$res a b c
lsort -integer	Sorts list numerically (not lexicographically) for correct min/max

TCL Script:

```

proc signal_analysis {times} {
    if {[llength $times] == 0} {
        error "Empty signal list!"
    }
    set sorted [lsort -integer $times]
    set min    [lindex $sorted 0]
    set max    [lindex $sorted end]
    set count  [llength $times]
    set sum    0
    foreach t $times { set sum [expr {$sum + $t}] }
    set avg    [expr {double($sum) / $count}]
    return [list $min $max $avg $count]
}

set clk_transitions {10 20 30 40 50 60 70 80 90 100}
set result [signal_analysis $clk_transitions]

lassign $result min max avg count

puts "Signal Transition Analysis:"
puts "-----"
puts "  Count    : $count"
puts "  Min      : $min ns"
puts "  Max      : $max ns"
puts "  Average  : [format %.2f $avg] ns"

```

Sample Output:

```

Signal Transition Analysis:
-----
Count      : 10
Min        : 10 ns
Max        : 100 ns

```

Average : 55.00 ns

Q9 Reusable Testbench Logger Proc

Write a reusable 'logger' proc that accepts a severity level (INFO, WARNING, ERROR) and a message. It should prefix each message with a timestamp counter, color-code the severity label, and maintain a global count of warnings and errors.

Key Concepts: global variables, upvar, proc reuse, incr, format

global	Allows a proc to read/write variables in the global scope
incr	Atomically increments an integer variable by 1 (or a custom step)
format %s	Formats strings with fixed-width padding for aligned output

TCL Script:

```
# Global counters
set log_time 0
set warn_count 0
set err_count 0

proc logger {severity message} {
    global log_time warn_count err_count
    incr log_time 10

    switch $severity {
        "INFO"      { set tag "[ INFO  ]" }
        "WARNING"   { set tag "[WARNING]" ; incr warn_count }
        "ERROR"     { set tag "[ ERROR ]" ; incr err_count }
        default     { set tag "[ ???  ]" }
    }

    puts "[format {%6d ns} $log_time] $tag $message"
}

proc log_summary {} {
    global warn_count err_count
    puts ""
    puts "====="
    puts " Log Summary"
    puts "====="
    puts " Warnings : $warn_count"
    puts " Errors   : $err_count"
    if {$err_count > 0} {
        puts " Status   : SIMULATION FAILED"
    } else {
        puts " Status   : SIMULATION PASSED"
    }
    puts "====="
}

logger INFO      "Reset applied to DUT"
logger INFO      "Clock started at 100MHz"
logger WARNING   "Data bus saw X-state at time 30ns"
```

```
logger INFO      "Sending first packet"
logger ERROR     "Assertion FAILED: req_ack_check"
logger WARNING   "Timeout on response - retrying"
logger INFO      "Simulation complete"

log_summary
```

Sample Output:

```
10 ns [ INFO ] Reset applied to DUT
20 ns [ INFO ] Clock started at 100MHz
30 ns [WARNING] Data bus saw X-state at time 30ns
40 ns [ INFO ] Sending first packet
50 ns [ ERROR ] Assertion FAILED: req_ack_check
60 ns [WARNING] Timeout on response -- retrying
70 ns [ INFO ] Simulation complete

=====
Log Summary
=====
Warnings : 2
Errors   : 1
Status   : SIMULATION FAILED
=====
```

SECTION 4 : List Operations

Q10 List Filtering & Sorting

Given a list of testcase names, write a TCL script to: sort them alphabetically, remove duplicates, filter only names containing 'sanity', and display the results in a formatted table.

Key Concepts: lsort, lsearch -all, index, foreach, string match

lsort -unique	Sorts a list alphabetically and removes duplicate entries
lsearch -all	Returns indices of all matching elements in a list
string match	Glob-style pattern match — supports * and ? wildcards

TCL Script:

```
set testcases {
    test_sanity_reset  test_mem_read    test_sanity_write
    test_overflow      test_sanity_reset test_alu_add
    test_sanity_clk    test_mem_write  test_alu_add
}

# Step 1: Sort and remove duplicates
set unique_sorted [lsort -unique $testcases]
puts "All Unique Tests ([llength $unique_sorted]):"
puts "-----"
foreach t $unique_sorted { puts "  $t" }

# Step 2: Filter tests containing "sanity"
set sanity_tests {}
foreach t $unique_sorted {
    if {[string match "*sanity*" $t]} {
        lappend sanity_tests $t
    }
}

puts ""
puts "Sanity Tests Only ([llength $sanity_tests]):"
puts "-----"
foreach t $sanity_tests { puts "  $t" }
```

Sample Output:

```
All Unique Tests (7):
-----
    test_alu_add
    test_mem_read
    test_mem_write
    test_overflow
    test_sanity_clk
    test_sanity_reset
    test_sanity_write

Sanity Tests Only (3):
```

```
-----
test_sanity_clk
test_sanity_reset
test_sanity_write
```

Q11 List Stack & Queue Operations

Implement a stack (LIFO) and a queue (FIFO) using TCL lists. Write push, pop, enqueue, and dequeue procs. Demonstrate using them to manage a pending testcase list.

Key Concepts: lappend, lindex, lreplace, lrange, llength

lappend	Appends to end of list — used for push (stack) and enqueue (queue)
lindex 0	Reads first element — used to peek at queue front
lreplace	Removes or replaces elements at given positions in a list

TCL Script:

```
# — Stack (LIFO) —————
proc stack_push {stack_var item} {
    upvar $stack_var stk
    lappend stk $item
    puts "PUSH: $item -> Stack: $stk"
}

proc stack_pop {stack_var} {
    upvar $stack_var stk
    if {[llength $stk] == 0} { puts "Stack EMPTY!"; return "" }
    set top [lindex $stk end]
    set stk [lreplace $stk end end]
    puts "POP : $top <- Stack: $stk"
    return $top
}

# — Queue (FIFO) —————
proc queue_enqueue {q_var item} {
    upvar $q_var q
    lappend q $item
    puts "ENQUEUE: $item -> Queue: $q"
}

proc queue_dequeue {q_var} {
    upvar $q_var q
    if {[llength $q] == 0} { puts "Queue EMPTY!"; return "" }
    set front [lindex $q 0]
    set q [lreplace $q 0 0]
    puts "DEQUEUE: $front <- Queue: $q"
    return $front
}

set mystack {}
puts "--- Stack Demo ---"
stack_push mystack "test_reset"
stack_push mystack "test_read"
```

```
stack_push mystack "test_write"
stack_pop  mystack
stack_pop  mystack

set myqueue {}
puts ""
puts "--- Queue Demo ---"
queue_enqueue myqueue "test_sanity"
queue_enqueue myqueue "test_overflow"
queue_enqueue myqueue "test_alu"
queue_dequeue myqueue
queue_dequeue myqueue
```

Sample Output:

```
--- Stack Demo ---
PUSH: test_reset  -> Stack: test_reset
PUSH: test_read   -> Stack: test_reset test_read
PUSH: test_write  -> Stack: test_reset test_read test_write
POP : test_write  <- Stack: test_reset test_read
POP : test_read   <- Stack: test_reset

--- Queue Demo ---
ENQUEUE: test_sanity    -> Queue: test_sanity
ENQUEUE: test_overflow  -> Queue: test_sanity test_overflow
ENQUEUE: test_alu       -> Queue: test_sanity test_overflow test_alu
DEQUEUE: test_sanity    <- Queue: test_overflow test_alu
DEQUEUE: test_overflow  <- Queue: test_alu
```

SECTION 5 : String Operations

Q12 Net Name Parser & Validator

Write a TCL script that takes a hierarchical net name like 'top/core/alu/sum[7:0]' and extracts: the full hierarchy path, the signal name, and the bit index. Also validate that the signal follows naming rules (lowercase, no spaces).

Key Concepts: string last, string range, regexp, string tolower

string last	Finds the last occurrence of a substring (useful for final '/')
string range	Extracts a substring between two character indices
regexp match	Captures named groups from a string using parentheses

TCL Script:

```
proc parse_net {net} {
    puts "====="
    puts " Parsing: $net"
    puts "====="

    # Extract bit index if present
    if {[regexp {(.*)(\[.*\])$} $net match base bitidx]} {
        puts " Bit Index : $bitidx"
    } else {
        set base $net
        set bitidx "none"
        puts " Bit Index : none (scalar signal)"
    }

    # Extract hierarchy and signal name
    set last_slash [string last "/" $base]
    if {$last_slash >= 0} {
        set hierarchy [string range $base 0 [expr {$last_slash - 1}]]
        set signame [string range $base [expr {$last_slash + 1}] end]
    } else {
        set hierarchy "top-level"
        set signame $base
    }
    puts " Hierarchy : $hierarchy"
    puts " Signal : $signame"

    # Validate: signal must be lowercase, no spaces
    if {[regexp {[A-Z ]} $signame]} {
        puts " Validation: FAIL (uppercase or spaces found)"
    } else {
        puts " Validation: PASS"
    }
    puts "====="
    puts ""
}

parse_net "top/core/alu/sum[7:0]"
```

```

parse_net "top/mem_ctrl/addr[31:0]"
parse_net "top/fetch/clock"
parse_net "top/decode/BadSignal"

```

Sample Output:

```

=====
Parsing: top/core/alu/sum[7:0]
=====
Bit Index : [7:0]
Hierarchy : top/core/alu
Signal    : sum
Validation: PASS
=====

=====
Parsing: top/decode/BadSignal
=====
Bit Index : none (scalar signal)
Hierarchy : top/decode
Signal     : BadSignal
Validation: FAIL (uppercase or spaces found)
=====

```

Q13 String-Based Report Formatter

Write a TCL proc that takes a test result list (each entry: testname status runtime_ns) and prints a neatly formatted report using string operations. Highlight PASS in one format and FAIL in another. Compute total runtime.

Key Concepts: format, string repeat, string toupper, append, index

format %-Ns	Left-aligns text in a field of N characters width
string repeat	Repeats a string N times — useful for drawing separator lines
append	Concatenates strings efficiently without creating new variables

TCL Script:

```

# Each entry: {testname status runtime_ns}
set results {
    {test_reset      PASS  120}
    {test_read       FAIL  340}
    {test_write      PASS  210}
    {test_overflow   FAIL  450}
    {test_sanity_clk PASS   95 }
}

set sep [string repeat "-" 50]
puts $sep
puts "[format %-22s {Test Name}][format %-8s Status] Runtime"
puts $sep

set total_rt  0
set pass_cnt  0
set fail_cnt  0

```

```

foreach entry $results {
    set name [lindex $entry 0]
    set stat [lindex $entry 1]
    set rt   [lindex $entry 2]
    incr total_rt $rt

    if {$stat eq "PASS"} {
        incr pass_cnt
        set tag "  PASS "
    } else {
        incr fail_cnt
        set tag "[ FAIL]"
    }
    puts "[format %-22s $name]$tag    ${rt} ns"
}

puts $sep
puts "Total Runtime : ${total_rt} ns"
puts "Passed       : $pass_cnt | Failed: $fail_cnt"
puts $sep

```

Sample Output:

```

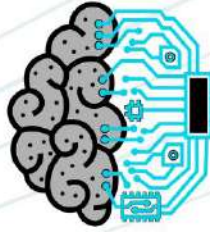
-----
Test Name           Status   Runtime
-----
test_reset          PASS     120 ns
test_read            [ FAIL]  340 ns
test_write           PASS     210 ns
test_overflow        [ FAIL]  450 ns
test_sanity_clk      PASS      95 ns
-----

Total Runtime : 1215 ns
Passed       : 3 | Failed: 2
-----

```

SUMMARY TABLE — All 13 Questions

Q#	Topic	Title	Key TCL Concepts
Q1	File I/O & Log Parsing	Read & Count Lines	file exists, open, gets, while
Q2	File I/O & Log Parsing	Parse Errors & Warnings	regexp, string tolower, file write
Q3	File I/O & Log Parsing	Extract Timestamps	regexp capture, lappend, lindex
Q4	Arrays	Signal State Tracker	array set, upvar, array names
Q5	Arrays	Multi-Run Regression Tracker	2D array keys, lsort -unique
Q6	Arrays	Array Statistics	array get, array size, double()
Q7	Procs	Default Args & Error Handling	default args, catch, error
Q8	Procs	Proc Returning Multiple Values	return list, lassign, lsort -integer
Q9	Procs	Reusable Logger Proc	global, incr, switch, format
Q10	List Operations	Filtering & Sorting	lsort -unique, string match
Q11	List Operations	Stack & Queue	lappend, lreplace, lindex, upvar
Q12	String Operations	Net Name Parser & Validator	string last/range, regexp
Q13	String Operations	Report Formatter	format, string repeat, append



TM

KT SEMICON

Technology for Tomorrow



www.ktsemicon.com



6303430469, 8977276692



info@ktsemicon.com



H.No.5-5-35/18, 3rd Floor, Near
Balanagar Metro Station, Prashanthi
Nagar, IDA Kukatpally, Kukatpally,
Hyderabad, Telangana 500018

Do Follow for more Updates

